A Comprehensive Performance Analysis of Apache Hadoop and Apache Spark *

N. Ahmed , Andre L. C. Barczak , Teo Susnjak, Mohammed A. Rashid

İnan Deniz Bal 333078004

Gökay Gülsoy 323078045

* Article can be found at https://doi.org/10.1186/s40537-020-00388-5

Why We Need Frameworks: The Big Data Problem

MASSIVE VOLUMES OF COMPLEX DATA

HARD FOR CONVENTIONAL TOOLS

DISTRIBUTED COMPUTING SYSTEMS

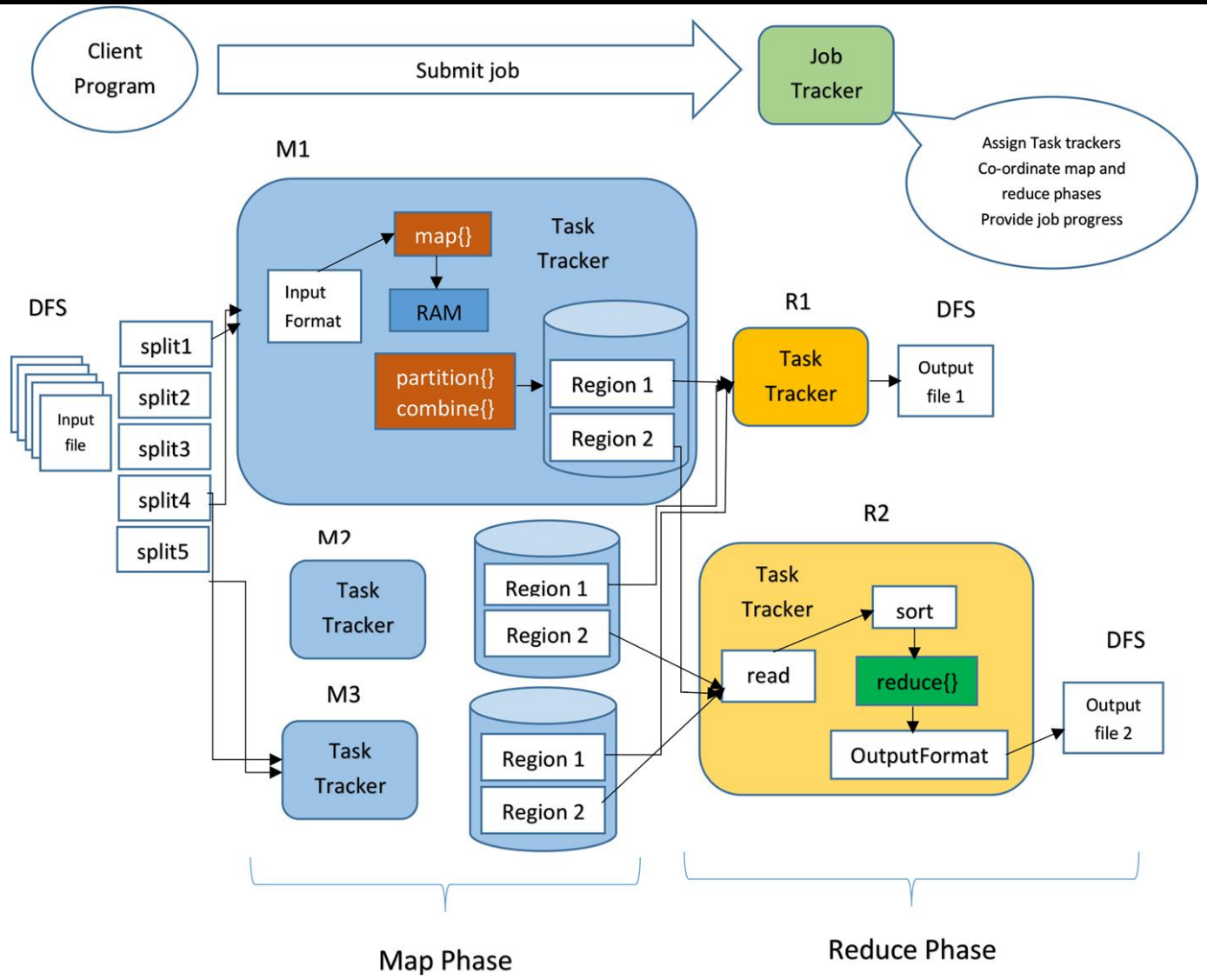
Hadoop System

Hadoop is an open source framework based on Java that manages the storage and processing of large amounts of data for applications.

MapReduce : Parallel processing and designed purposely to write, read, and process bulky amounts

Map Phase + Shuffle Phase(?) + Reduce Phase = Your lovely Hadoop

Hadoop Workflow



Spark

DESIGNED BASED HADOOP, OPEN-SOURCE CLUSTER COMPUTING FRAMEWORK

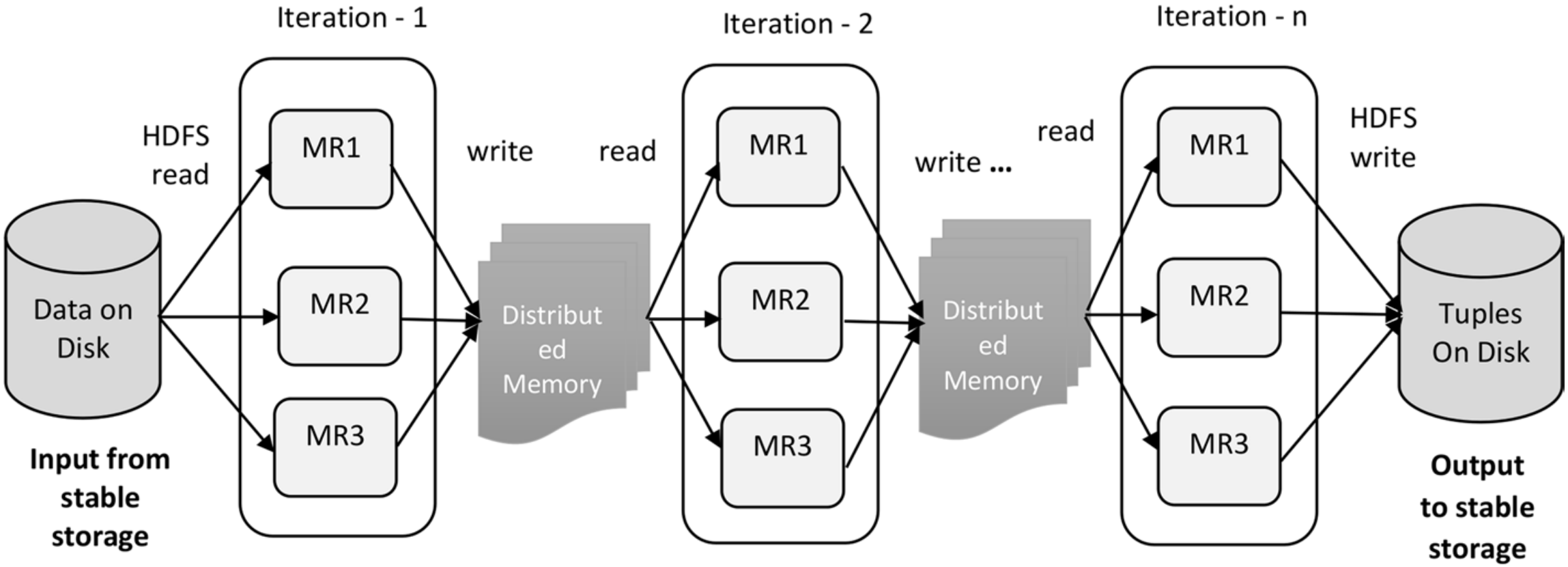
«FITS A WIDER CLASS OF APPLICATIONS THAN MAPREDUCE WHILE MAINTAINING THE AUTOMATIC FAULT TOLERANCE»

SUPPORTS MLIB1, GRAPHX, AND SPARK STREAMING FOR BIG DATA

FAST: USES MEMORY TO STORE DATA REDUCING THE READ/WRITE CYCLE

RESILIENT DISTRIBUTED DATASETS (RDD) AND DIRECTED ACYCLIC GRAPH (DAG)

Spark Workflow



Related Works : HM

They have concluded that the performance of Spark is much better in most cases as compared to Hadoop.

They have concluded that the Spark speed reaches up to three times higher than the MapReduce, though Spark performance heavily depends on sufficient memory size

Shuffle Behavior, Compression and Serialization, and Memory Management : SPARK IS SUPPERIOR

Experiment Parameters

- 600 GB of data generated with string concatenation technique
- *WordCount*: **map-dependent aggregation job**
- *TeraSort*: **shuffle job**

Table 2 Experimental Hadoop cluster

Server configuration	Processor	2.9 GHz
	Main Memory	64 GB
	Local Storage	10 TB
Node configuration	CPU	Intel(R) Xeon(R) CPU E3-1231 v3 @ 3.40GHz
	Main Memory	32 GB
	Number of Nodes	10
	Local Storage	6 TB each, 60 TB total
	CPU cores	8 each, 80 total
Software	Operating System	Ubuntu 16.04.2 (GNU/Linux 4.13.0-37-generic×86 64)
	JDK	1.7.0
	Hadoop	2.4.0
	Spark	2.1.0
Workload	Micro Benchmarks	WordCount, and TeraSort

Hadoop Parameters

Table 3 Hadoop configuration parameters

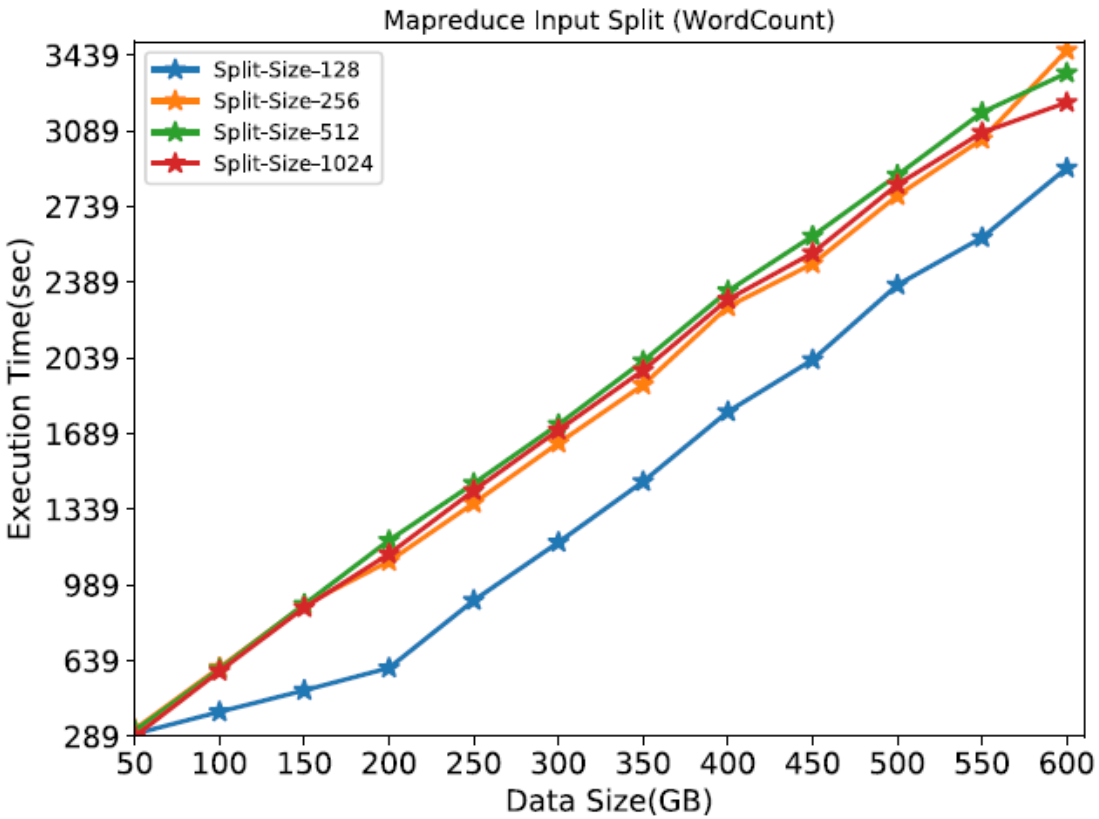
Configuration parameters category	Hadoop	Tuned values
Resource utilization	mapreduce.reduce.memory	8 GB
	mapred.reduce.task	16,384 MB, 25,600 MB
	mapreduce.reduce.cpu.vcores	4
Input split	mapred.min.split.size, mapred.max.split.size	128 MB (default), 256 MB, 512 MB, 1024 MB
Shuffle	i/o.sort.mb	25, 50, 75, 100
	i/o.sort.factor	512, 1024, 1536, 2047
	mapreduce.reduce.shuffle.parallelcopies	50, 100, 150, 200
	mapreduce.task.io.sort.factor	15, 30, 45, 60

Spark Parameters

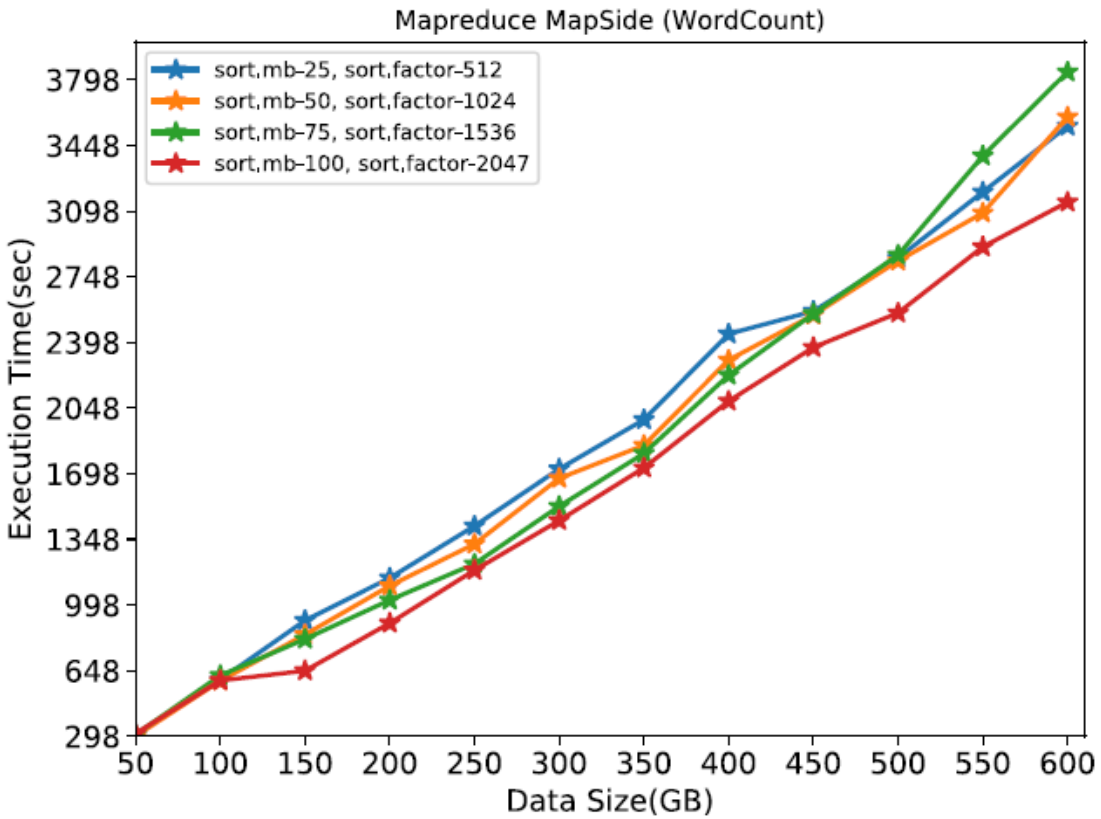
Table 4 Spark configuration parameters

Configuration parameters category	Spark	Tuned values
Resource utilization	num-executors	50
	executor-cores	4
	executor-memory	8 GB
Input split	spark.hadoop.MapReduce.input .fileinputformat.split.minsize	128 MB (default), 256MB, 512MB, 1024MB
Shuffle	spark.shuffle.file.buffer	16 k, 32 k (default), 48 k, 64 k
	spark.reducer.maxSizeInFlight	32 M, 48 M (default), 64 M, 96 M
	spark.hadoop.dfs.replication	1
	spark.default.parallelism	80, 100, 200, 300

The performance of the WordCount application for Hadoop

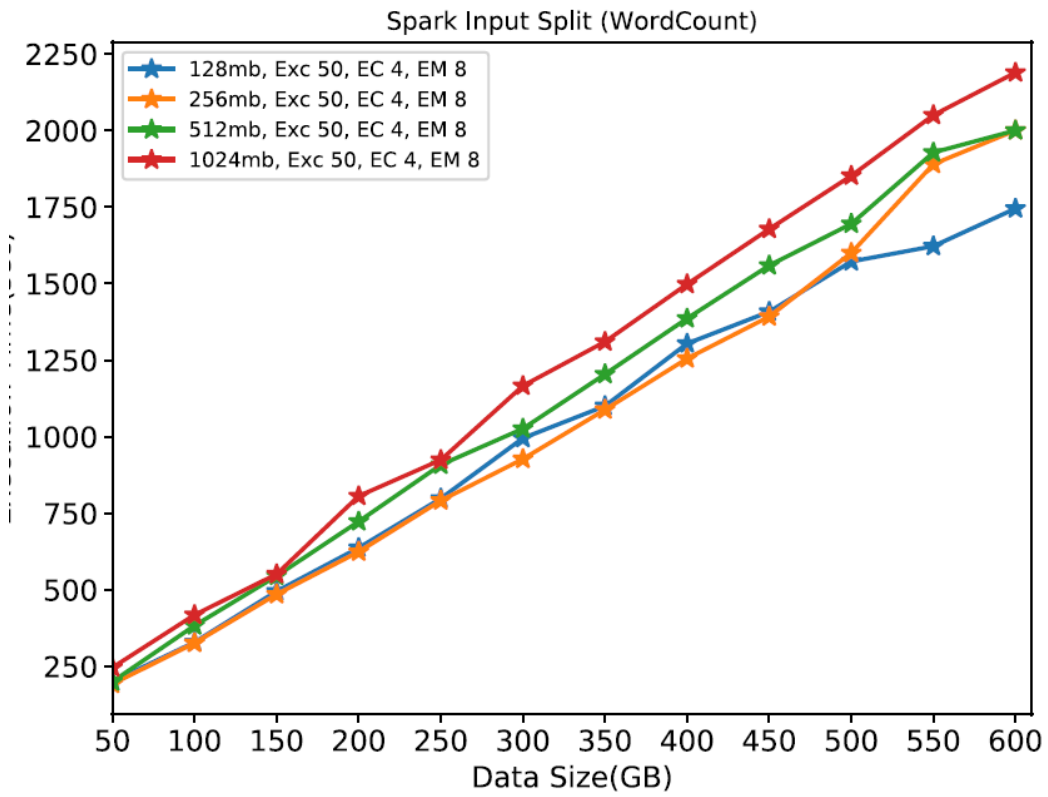


a

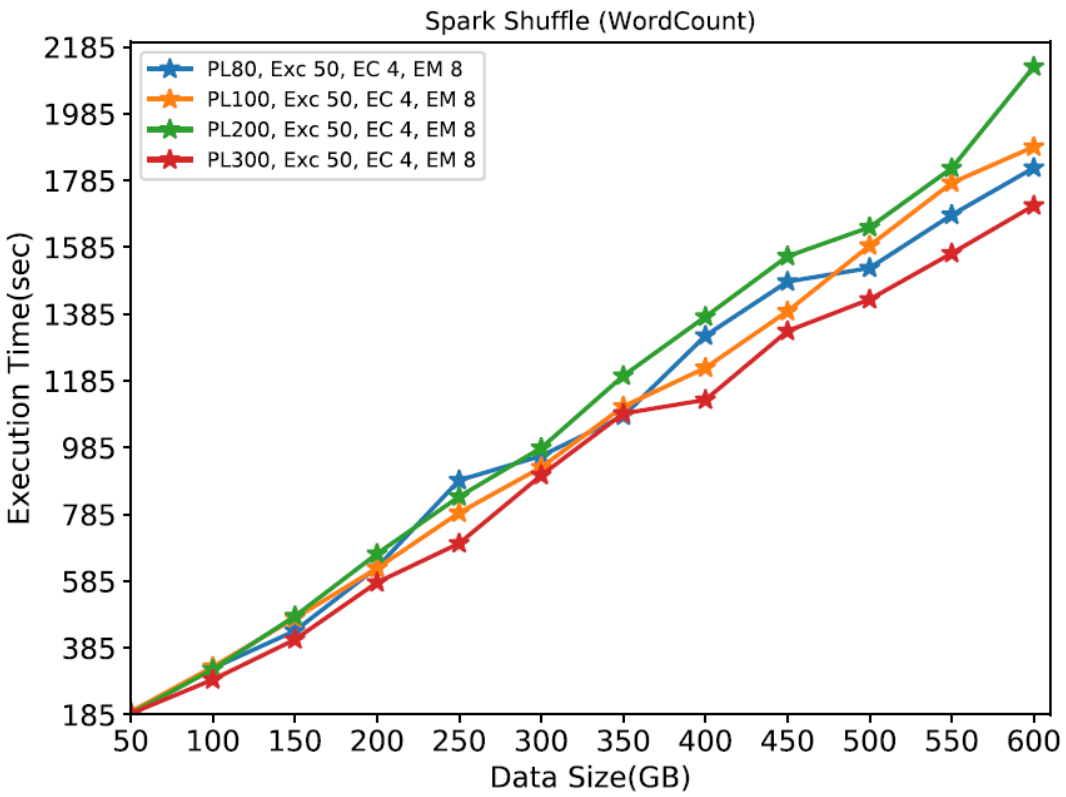


b

The performance of the WordCount application for Spark



c



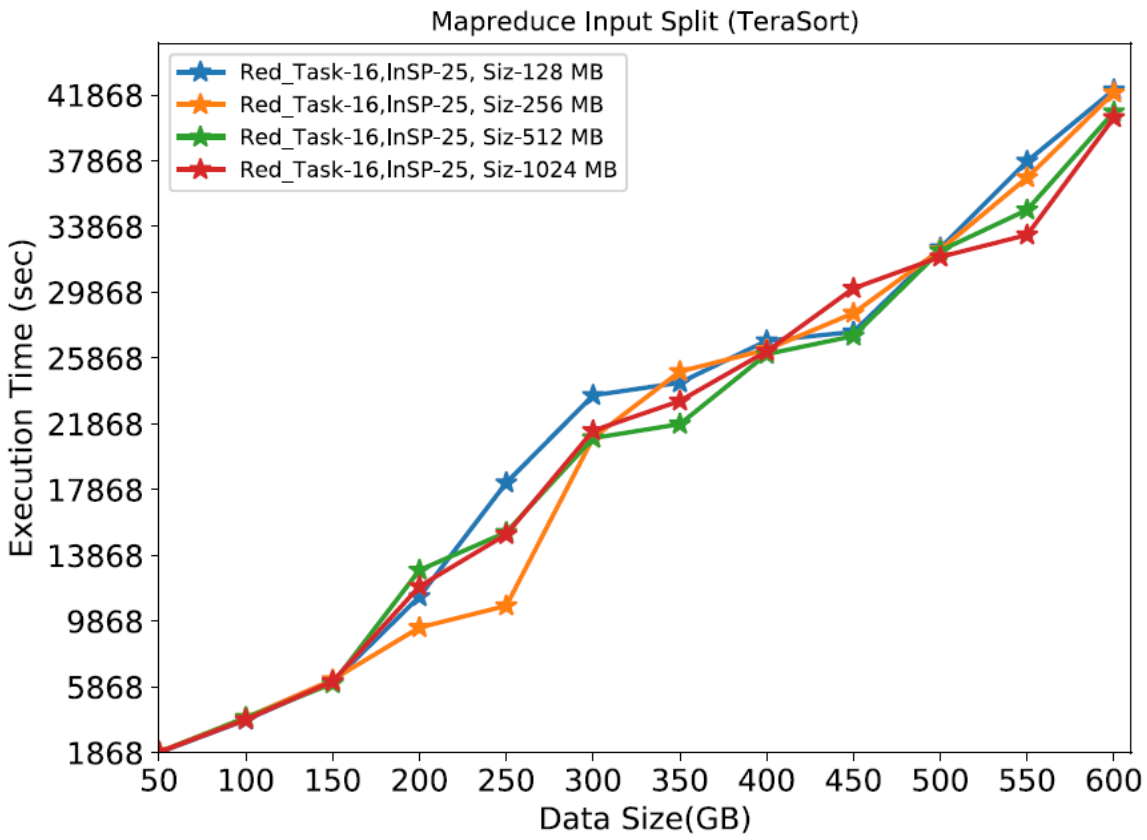
d

Test Results

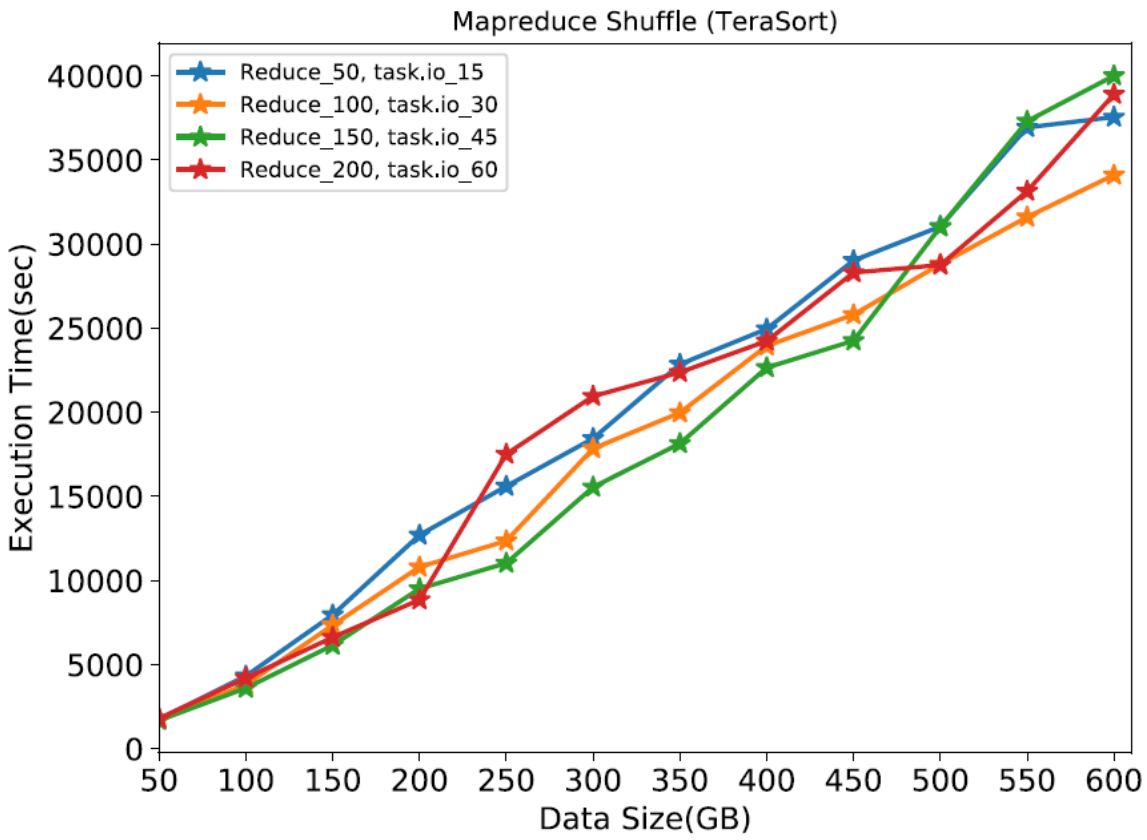
Table 5 The best execution time of MapReduce and Spark with WordCount workload

	Split sizes (MB)	Execution time (s)
MapReduce input splits (WordCount)	128	2376
Spark input splits (WordCount)	256	1392
MapReduce shuffle (WordCount)	100	2371
Spark shuffle (WordCount)	300	1334

The performance of the TerraSort application for Hadoop

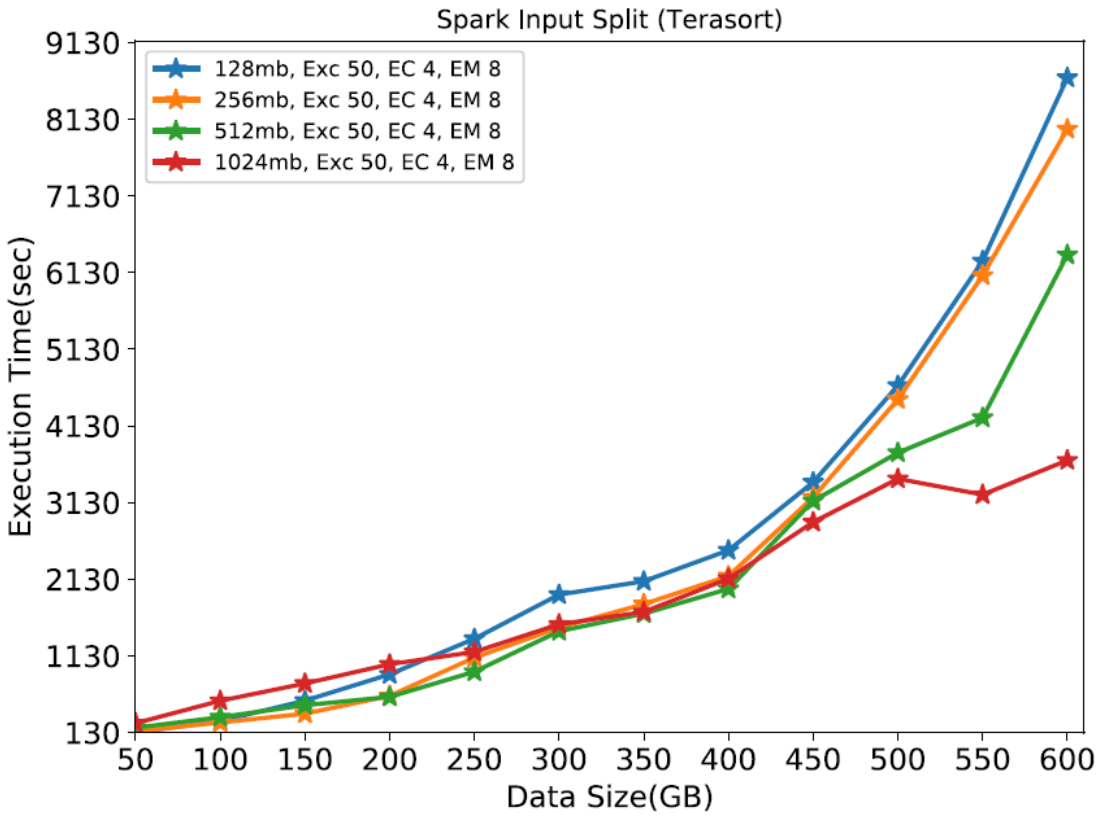


a

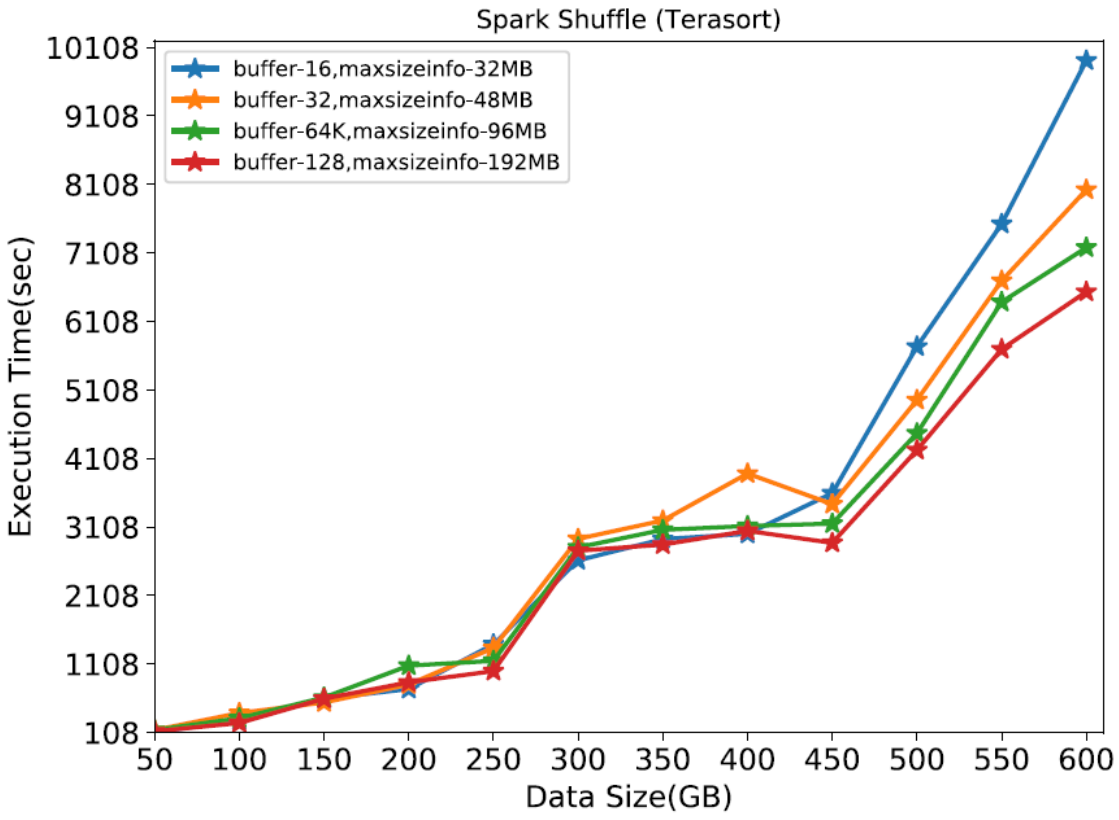


b

The performance of the TerraSort application for Spark



c



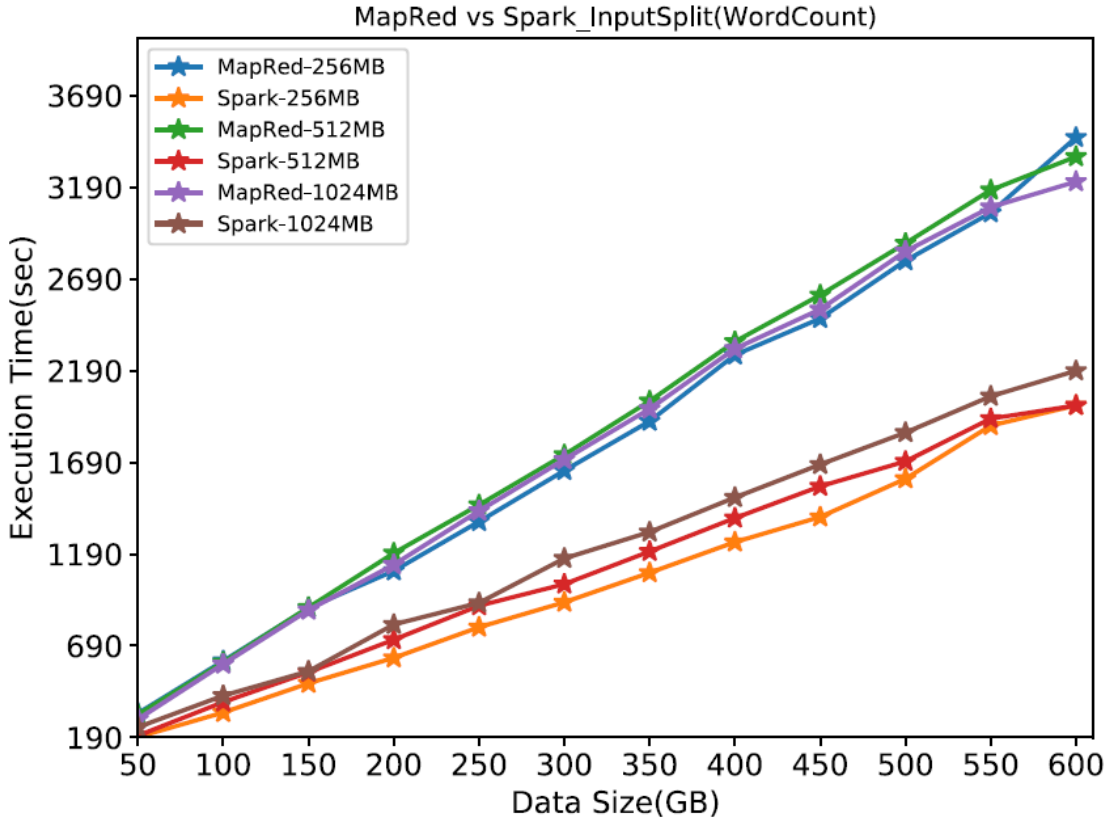
d

Test Results

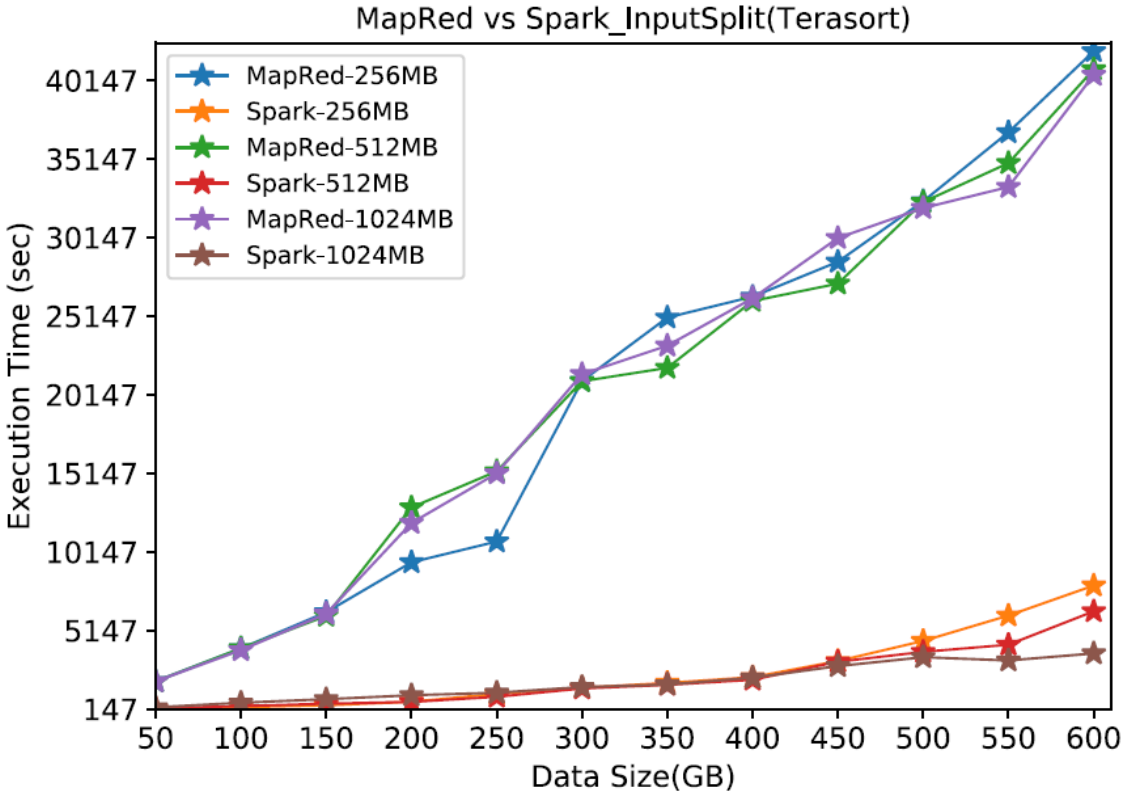
Table 6 The best execution time of MapReduce and Spark with Terasort workload

	Split sizes (MB)	Execution time (s)
MapReduce input splits (TeraSort)	256	21,014
Spark input splits (TeraSort)	512 & 1024	3780 & 3439
MapReduce shuffle (TeraSort)	150 & 45	24,250
Spark shuffle (TeraSort)	128 & 192	6540

The comparison of Hadoop and Spark with WordCount and TeraSort

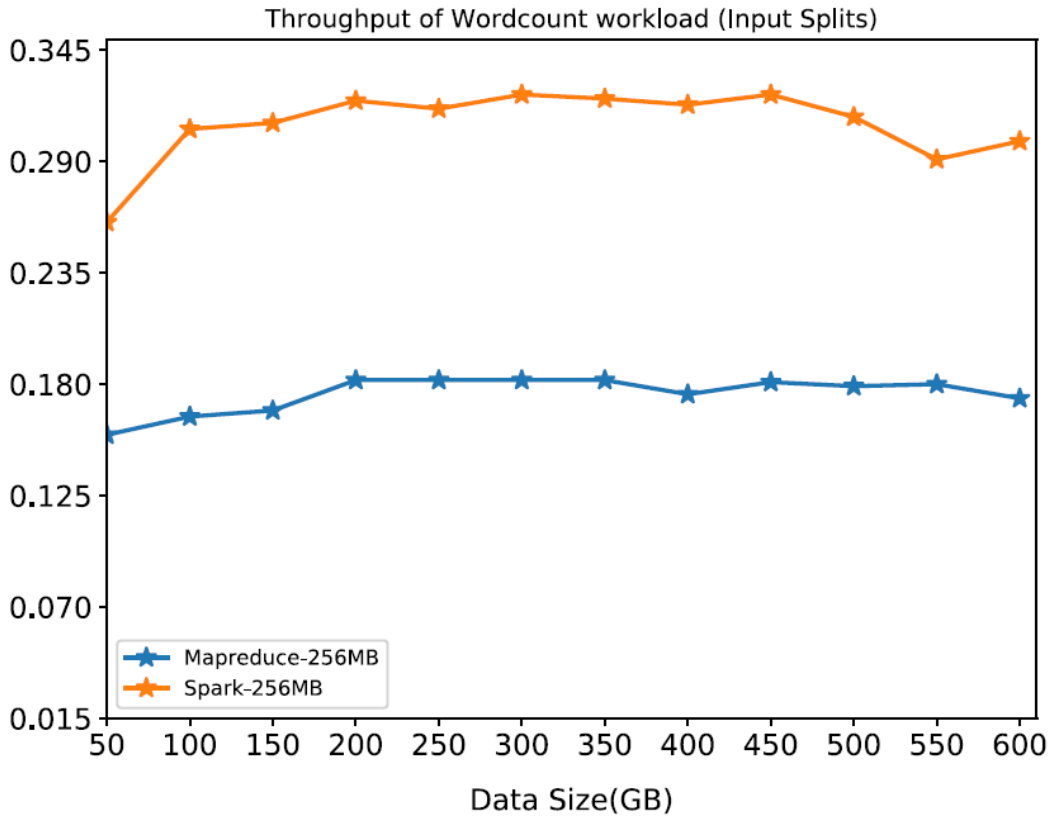


a

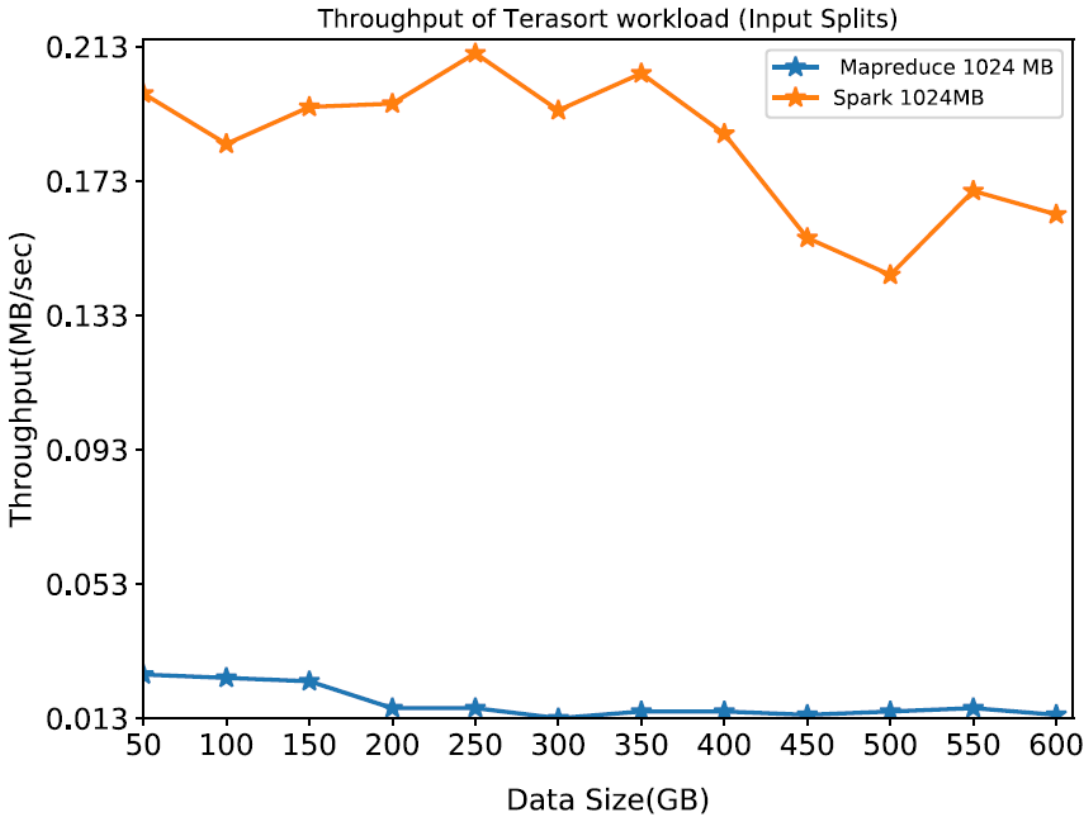


b

Throughput of WordCount and TeraSort workload

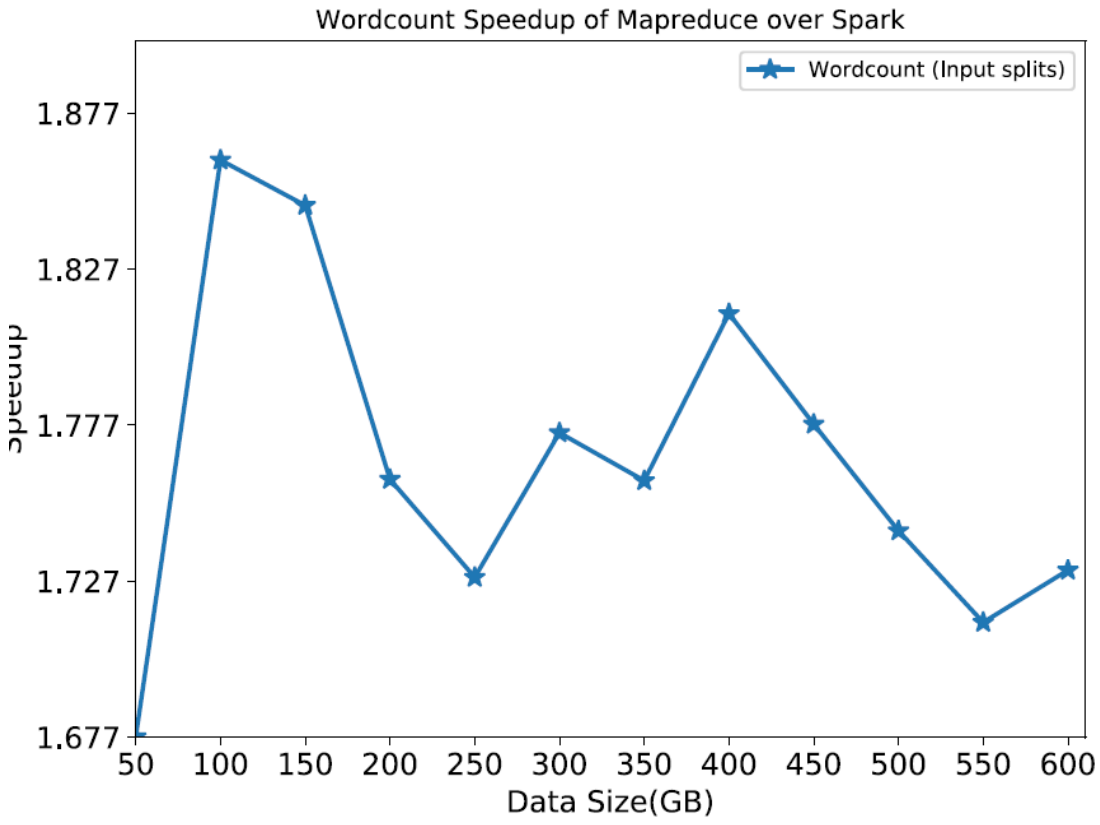


a

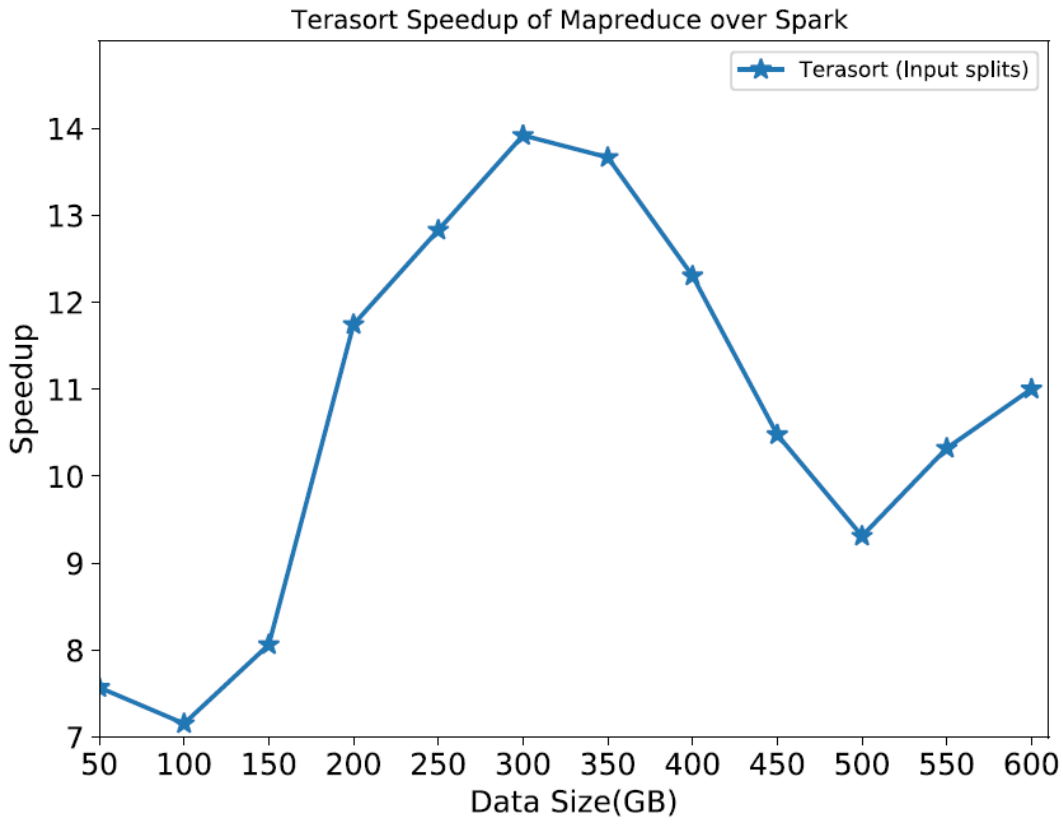


b

Speedup Results

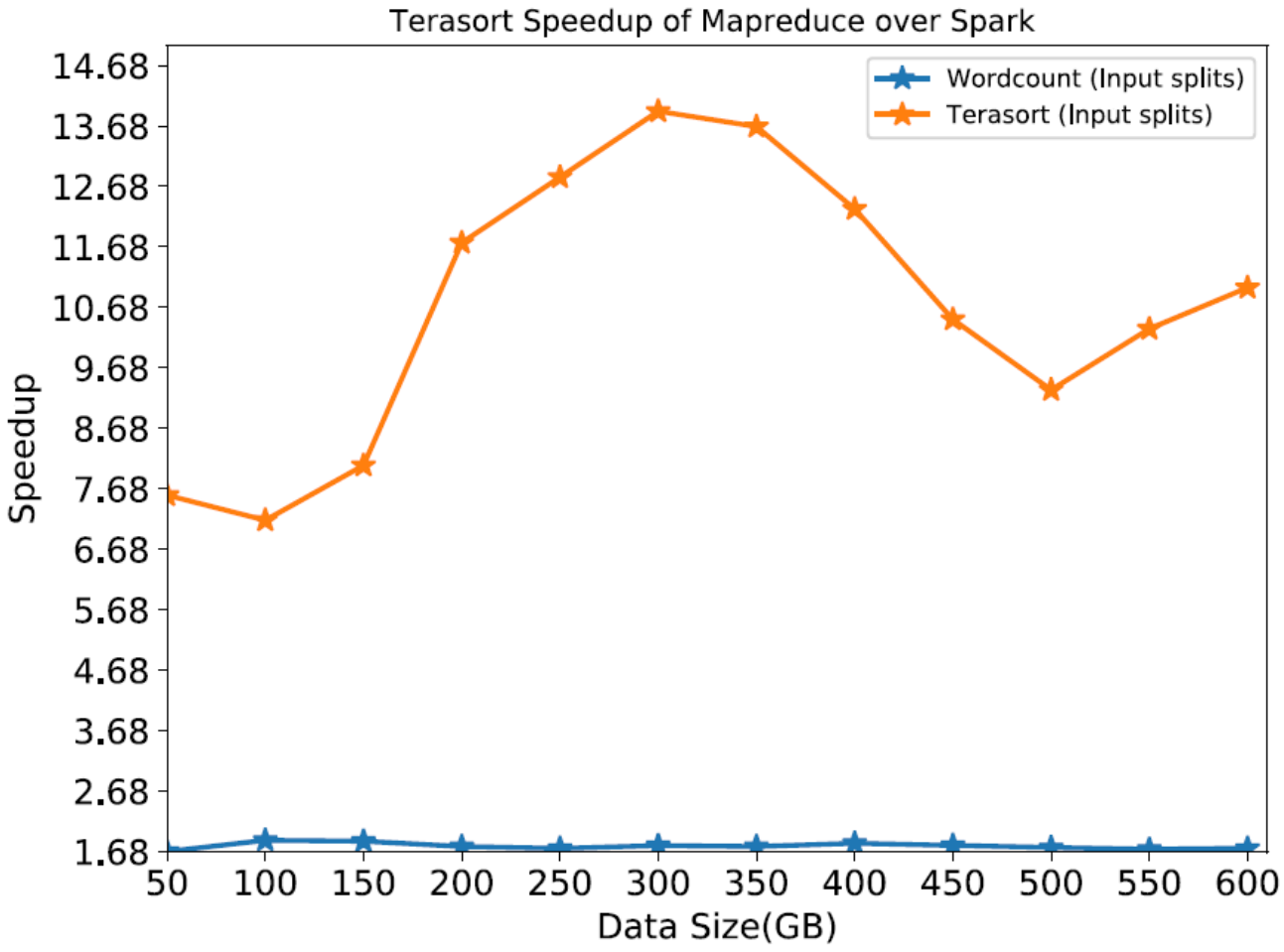


a



b

Terrasort Speedup of Mapreduce over Spark



c

Conclussion

Hadoop and **Spark** systems performance heavily **depends on input data size** and right parameter selection and **tuning**.

Spark has better performance as compared to **Hadoop** by **2 times** with WordCount work load and **14 times** with Tera-Sort workloads respectively

Throughput and speedup results show that **Spark** is more **stable** and **faster** than **Hadoop**

Spark data processing ability in memory instead of store in disk for the map and reduced function => **FASTER**

Thanks for listening

Questions